PROJECT:
CHESS GAME

Made by : Samreen Zia

Semester : 2A

Submitted to : Dr. Sobia Khalid

Program:

```
#include "stdafx.h"
#include<iostream>
#include<string>
using namespace std;
enum piecetype{king,queen,bishop,rock,knight,pawn};
enum piececolor{white,black};
bool gameOver = false;
string winner ="";
class Piece{
public:
       piecetype type;
       piececolor color;
public:
       Piece( piecetype t,piececolor c):type(t),color(c)
       {}
       virtual bool isValidMove(int x1, int y1, int x2, int y2) = 0;

       void display() {
        switch (type) {
            case king:
                cout << (color == white ? "WK" : "BK");
                break;
            case queen:
                cout << (color == white ? "WQ" : "BQ");
                break;
            case rock:
                cout << (color == white ? "WR" : "BR");
                break;
            case bishop:
                cout << (color == white ? "WB" : "BB");
                break;
            case knight:
                cout << (color == white ? "WN" : "BN");
                break;
            case pawn:
                cout << (color == white ? "WP" : "BP");
                break;
        }
    }
};
class King : public Piece {
public:
    King(piececolor c) : Piece(king , c) {}

    bool isValidMove(int x1, int y1, int x2, int y2) {
        return abs(x2 - x1) <= 1 && abs(y2 - y1) <= 1;
    }
};
class Queen : public Piece {
public:
    Queen(piececolor c) : Piece(queen, c) {}

    bool isValidMove(int x1, int y1, int x2, int y2) {
        return x1 == x2 || y1 == y2 || abs(x2 - x1) == abs(y2 - y1);
    }
```

```
};
class Rock : public Piece {
public:
    Rock(piececolor c) : Piece(rock, c) {}

    bool isValidMove(int x1, int y1, int x2, int y2) {
        return x1 == x2 || y1 == y2;

    }
};
class Bishop : public Piece {
public:
    Bishop(piececolor c) : Piece(bishop, c) {}

    bool isValidMove(int x1, int y1, int x2, int y2) {
        return abs(x2 - x1) == abs(y2 - y1);
    }
};
class Knight : public Piece {
public:
    Knight(piececolor c) : Piece(knight, c) {}

    bool isValidMove(int x1, int y1, int x2, int y2) {
        return (abs(x2 - x1) == 2 && abs(y2 - y1) == 1) || (abs(x2 - x1) == 1 && abs(y2 -
y1) == 2);
    }
};
class Pawn : public Piece {
public:
    Pawn(piececolor c) : Piece(pawn, c) {}

    bool isValidMove(int x1, int y1, int x2, int y2) {
        if (color==white) {
            return (x2 == x1 && y2 == y1 + 1) || (x2 == x1 && y1 == 1 && y2 == y1 +
2)||(abs(x2 - x1) == abs(y2 - y1))+1;
        } else {
            return (x2 == x1 && y2 == y1 - 1) || (x2 == x1 && y1 == 6 && y2 == y1 -
2)||(abs(x2 - x1) == abs(y2 - y1))-1;
        }
    }
};
class Square {
public:
    Piece* piece;

    Square(Piece* piece = nullptr) {
        this->piece = piece;
    }

    void occupy(Piece* piece) {
        this->piece = piece;
    }
       void display() {
        if (piece != nullptr) {
            piece->display();
        } else {
            std::cout << "--";
        }
```

```
    }
};
class Board {
public:
    Square squares[8][8];
    void initialize() {

        for (int i = 0; i < 8; i++) {
                    squares[1][i].occupy(new Pawn(white));
            squares[6][i].occupy(new Pawn( black));
        }


              squares[0][0].occupy(new Rock(white));
              squares[0][1].occupy(new Knight(white));
              squares[0][2].occupy(new Bishop(white));
              squares[0][3].occupy(new Queen(white));
              squares[0][4].occupy(new King(white));
              squares[0][5].occupy(new Bishop(white));
              squares[0][6].occupy(new Knight(white));
              squares[0][7].occupy(new Rock(white));

              squares[7][0].occupy(new Rock(black));
        squares[7][1].occupy(new Knight(black));
        squares[7][2].occupy(new Bishop(black));
        squares[7][3].occupy(new Queen(black));
        squares[7][4].occupy(new King(black));
              squares[7][5].occupy(new Bishop(black));
        squares[7][6].occupy(new Knight(black));
        squares[7][7].occupy(new Rock(black));
    }

    void display() {
              for (int j = 0; j < 8; j++) {
                     cout<<" "<<j<<"  ";}
              cout<<endl;
        for (int i = 0; i < 8; i++) {
            for (int j = 0; j < 8; j++) {
                squares[i][j].display();
                cout << "  ";
            }
            cout<<i<<endl;
        }

    }
};
class Player {
public:
    piececolor color;
       string name;
    Player(piececolor color,string name) {
        this->color = color;
              this->name = name;
    }
    bool makeMove(Board& board, int x1, int y1, int x2, int y2) {
    Piece* piece = board.squares[y1][x1].piece;
       if (gameOver) {
    cout << "Game is over. No more moves allowed." << endl;
```

```
    return false;
}

    if (piece != nullptr && piece->color == color) {
        if (piece->isValidMove(x1, y1, x2, y2)) {
                        Piece* targetPiece = board.squares[y2][x2].piece;
            if (targetPiece != nullptr && targetPiece->type == king&& targetPiece->color
!= piece->color) {

                 delete board.squares[y2][x2].piece;
        board.squares[y2][x2].occupy(piece);
                board.squares[y1][x1].piece = nullptr;
                            gameOver = true;
                            winner = (color == white) ? "White" : "Black";
                return true;
            }
            delete board.squares[y2][x2].piece;
            board.squares[y2][x2].occupy(piece);
            board.squares[y1][x1].piece = nullptr;
            cout << "Move made successfully!" << endl;
                     return true;
        }
              else {
            cout << "Invalid move!Try again" << endl;
        }
    }
       else  {
        cout << "You don't have a piece there or it's not your turn!Try again" << endl;
    }
        return false;
}

};
int main() {
    Board board;
    board.initialize();
       board.display();
     Player whitePlayer(white,"white");
    Player blackPlayer(black,"black");
    piececolor currentPlayer = white;

    while (true) {
        int x1, y1, x2, y2;
        cout << "Enter the coordinates of the piece you want to move (x y): ";
        cin >> x1 >> y1;
        cout << "Enter the coordinates where you want to move the piece (x y): ";
        cin >> x2 >> y2;

         bool moveMade = false;
        if (currentPlayer == white) {
            moveMade = whitePlayer.makeMove(board, x1, y1, x2, y2);
            if (moveMade){ currentPlayer = black;
                     cout<<"black's turn"<<endl;}
        } else {
            moveMade = blackPlayer.makeMove(board, x1, y1, x2, y2);
            if (moveMade) {currentPlayer = white;
                     cout<<"white's turn"<<endl;}
        }
```

```
            board.display();
             if (gameOver) {
          cout << "Game over! A king has been captured."<< endl;
                  cout<<winner<<"wins"<<endl;
          break;
      }
   }

      system("pause");
   return 0;
}
```

Output:

- ✔ W= white, B = black, R= rock, N= knight, Q= queen, K = king, P= pawn.
- ✔ All pieces and their coordinates are being displayed.
- ✔ White will add coordinates for the piece to move and then it will shift to black.

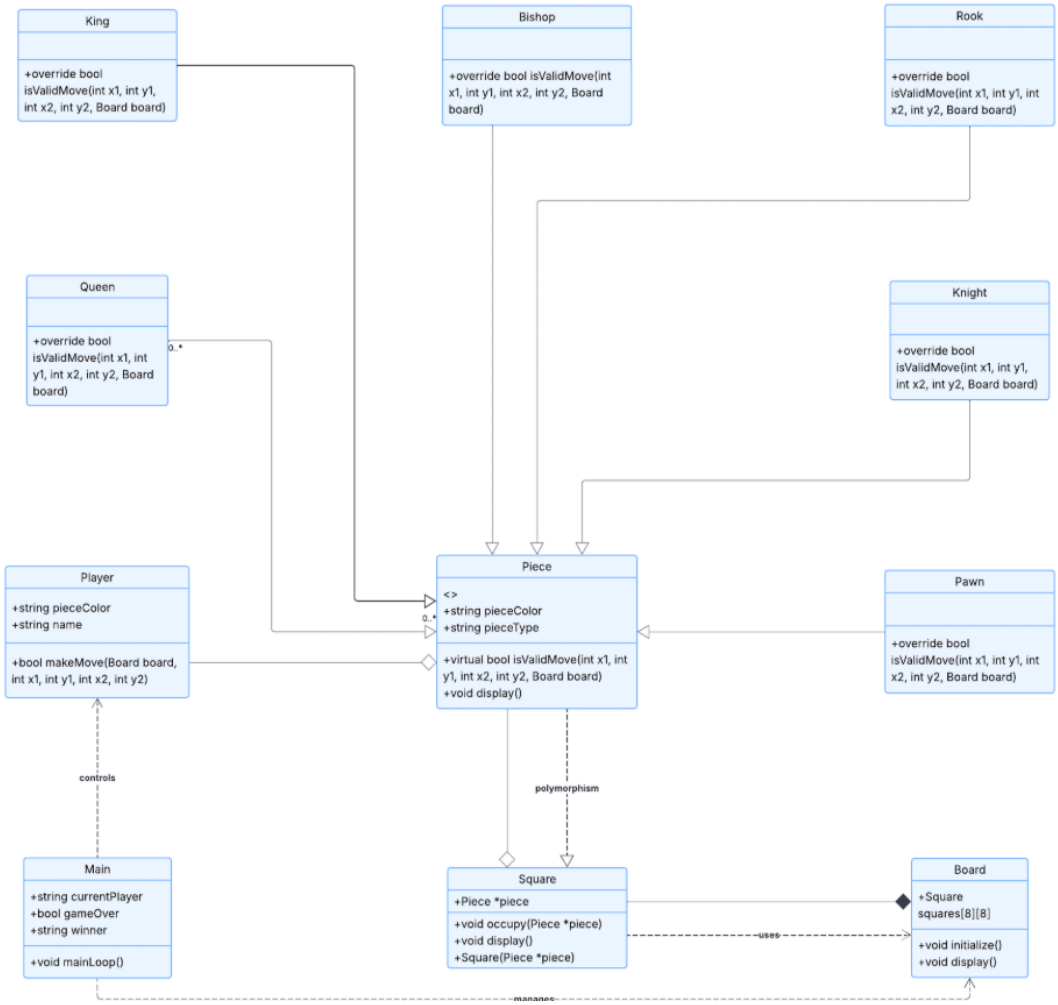
White Player:

```
 0   1   2   3   4   5   6   7
WR  WN  WB  WQ  WK  WB  WN  WR  0
WP  WP  WP  WP  WP  WP  WP  WP  1
--  --  --  --  --  --  --  --  2
--  --  --  --  --  --  --  --  3
--  --  --  --  --  --  --  --  4
--  --  --  --  --  --  --  --  5
BP  BP  BP  BP  BP  BP  BP  BP  6
BR  BN  BB  BQ  BK  BB  BN  BR  7
Enter the coordinates of the piece you want to move (x y): 0
1
Enter the coordinates where you want to move the piece (x y): 0
3
Move made successfully!
```

Black Player:

```
black's turn
 0   1   2   3   4   5   6   7
WR  WN  WB  WQ  WK  WB  WN  WR  0
--  WP  WP  WP  WP  WP  WP  WP  1
--  --  --  --  --  --  --  --  2
WP  --  --  --  --  --  --  --  3
--  --  --  --  --  --  --  --  4
--  --  --  --  --  --  --  --  5
BP  BP  BP  BP  BP  BP  BP  BP  6
BR  BN  BB  BQ  BK  BB  BN  BR  7
Enter the coordinates of the piece you want to move (x y): 0
6
Enter the coordinates where you want to move the piece (x y): 0
4
Move made successfully!
```

White Player:

```
white's turn
 0   1   2   3   4   5   6   7
WR  WN  WB  WQ  WK  WB  WN  WR  0
--  WP  WP  WP  WP  WP  WP  WP  1
--  --  --  --  --  --  --  --  2
WP  --  --  --  --  --  --  --  3
BP  --  --  --  --  --  --  --  4
--  --  --  --  --  --  --  --  5
--  BP  BP  BP  BP  BP  BP  BP  6
BR  BN  BB  BQ  BK  BB  BN  BR  7
Enter the coordinates of the piece you want to move (x y):
```

UML Class Diagram